Technicien Spécialisé en Développement Informatique



Algorithmique

0021266950863 :Tel

Auteur : EL HAIDAOUI Youssef Ingenieur En informatique

Introduction



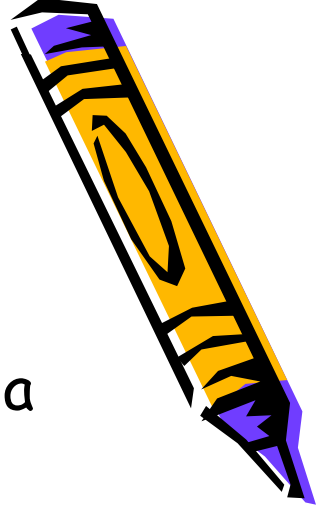
- Pour qu'un ordinateur fonctionne, il est nécessaire de lui dire **quoi faire**

- Toute action réalisée par une machine a été programmée par **un être humain.**

- un ordinateur **ne décide rien**, il fait "bêtement" ce qu'il lui a été programmé.



Mais qu'est-ce qu'un programme?



- ✓ Un programme est donc une suite d'instructions exécutées par la machine.

- ✓ La machine a son propre langage appelé langage machine.



Algorithme



- Est une suite de règles à appliquée dans un ordre déterminé.

- Est une suite d'instructions écrite en langage d'algorithme qui résout un problème et qui sont exécutable par n'importe qu'elle machine.

- Le rôle de l'algorithme est fondamental, on effet sans l'algorithme il n y aura pas de programme.



Méthodologie



La résolution d'un problème sur ordinateur est caractérisé par 4 étapes :

- Comprendre la nature du problème posé
- Préciser les données fournies (entrées)
- Préciser les résultas que l'on désire obtenir (Sorties)
- Déterminer le processus de transformation des données en résultats.



Comment on programme ?



nous allons utiliser un pseudo-langage, comportant toutes les structures de base d'un langage de programmation.

ensuite on traduit notre "pseudo" en langage évolué en fonction des possibilités de ce langage.

Ce langage sera ensuite traduit en langage machine.



Un programme est donc une suite d'instructions exécutées par la machine.



Ces instructions peuvent :

soit s'enchaîner les unes après les autres, on parle alors **de séquence d'instructions**;

ou bien s'exécuter dans certains cas et pas dans d'autres, on parle alors de **structure alternative**;

ou se répéter plusieurs fois, on parle alors de **structure répétitive**.



La séquence d'instructions



- Une instruction est une action que l'ordinateur est capable d'exécuter.

Une séquence d'instruction serait :

Se lever
Prendre sa douche
Prendre le petit déjeuner
S'habiller

Vous voyez que l'ordre des instructions a de l'importance : "S'habiller" puis "prendre sa douche" conduit à un résultat pas génial que nous appellerons un "bug".

Cependant certaines instructions peuvent se dérouler dans un ordre indifférent



structure alternative

Une alternative s'exprime par si ….. sinon……

Si fin de semaine ou congé
    Se lever
    Prendre son petit déjeuner
    Prendre sa douche
    Mettre sa tenue de sport
    Faire son jogging
Sinon
    Se lever
    Prendre son petit déjeuner
    Prendre sa douche
    Mettre sa tenue de travail
    Aller travailler
Fin Si



Optimisation

Que la condition soit réalisée (**condition vraie**) ou qu'elle ne le soit pas (**condition fausse**) les premières actions sont les mêmes et se passent dans le même ordre ce qui permet la simplification suivante :

afin de permettre une meilleure lisibilité; on parle d'**indentation**.

```
Se lever
Prendre son petit déjeuner
Prendre sa douche

Si fin de semaine ou congé
        Mettre sa tenue de sport
        Faire son jogging
Sinon
        Mettre sa tenue de travail
        Aller travailler
Fin Si
```



structure répétitive (itérative)

La routine journalière d'un employé est :

```
Ouvrir guichet
Appeler premier client

Tant que client dans file d'attente et pas fin de journée

        Traiter client
        Appeler client suivant

FinTantQue
```

Les deux actions "Traiter client" et "Appeler client suivant" vont se répéter tant que la condition située derrière l'instruction "Tant que" est vérifiée.



Considérons maintenant le programme complet de la journée

```
Se lever
Prendre son petit déjeuner
Prendre sa douche

Si fin de semaine ou congé
    Mettre sa tenue de sport
    Faire son jogging
    Passer une journée de détente
Sinon
    Mettre sa tenue de travail
    Aller travailler
    Faire travail
FinSi

Rentrer à la maison
Dîner
Aller se coucher
```



Considérons maintenant le programme complet de la journée (Suite)

```
Fonction travail
    Ouvrir guichet
    Appeler premier client
    Tant que pas heure de déjeuner
        Faire guichet
    FinTantQue
    Déjeuner
    Tant que client et pas heure de sortie
        Faire guichet
    FinTantQue
Fin Fonction

Fonction Guichet
    Si client en file d'attente
        Traiter client
        Appeler client suivant
    Sinon
        Classer
    FinSi
Fin Fonction
```



Explication

fonction Travail pour une meilleure lisibilité du programme.

On a créé une **fonction guichet** afin de ne pas répéter la même séquence d'instructions deux fois dans le programme.

Notre programme a donc été scindé en deux parties :

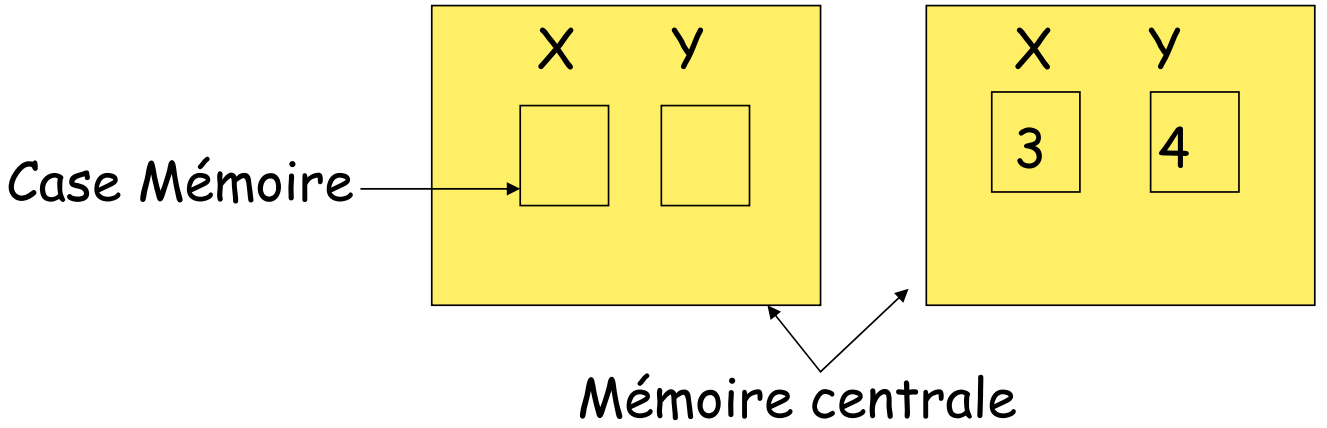
le corps du programme de la ligne 1 à la ligne 15
les fonctions ou sous-programmes internes à partir de la ligne 17.

Comment cela se passe-t-il lorsque nous rencontrons un appel de fonction ?



Les variables

Une variable est une zone mémoire qu'elle peut varier le contenu au cours de l'exécution du programme : son nom est appelé **Identificateur**



variable est caractérisée par :

- une adresse c'est à dire un emplacement dans la mémoire de la machine,
- un type permettant d'indiquer la nature de l'information contenue,
- éventuellement une longueur .

Les types de variables



Les variables que l'on utilise dans les programmes ne sont pas toutes de même nature

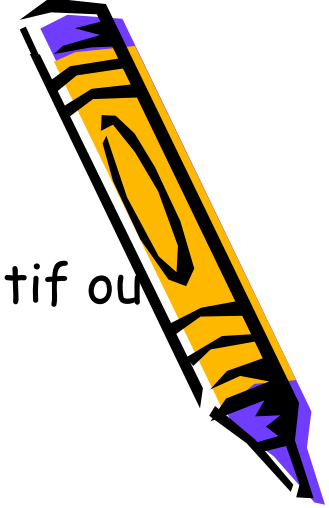
il y a des nombres, des caractères, ... On dit que les variables sont typées.

Il est nécessaire de donner un type aux variables, pour contrôler leur utilisation

(ex: on ne peut pas diviser un caractère par un entier)



Généralement les langages de programmation offrent les types suivants



entier : il s'agit des variables destinées à contenir un nombre entier positif ou négatif.

ENTIER variable, variable, ... ;

réel : il s'agit des variables numériques qui ne sont pas des entiers, c'est à dire qui comportent des décimales

REEL variable, variable, ... ;

caractère : Les variables de type caractère contiennent des caractères alphabétiques ou numériques (de 0 à 9)

CAR variable , variable , ...;

booléen : Il est souvent nécessaire lorsque l'on écrit un programme d'introduire des variables qui prennent les valeurs **vrai** ou **faux** ou les valeurs **oui** ou **non**.

BOOLEEN variable, variable, ... ;



Les opérateurs



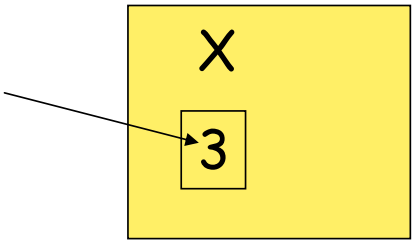
- Arithmétique : +; /; -; *; mode
- Comparaison : <; <=; >; >=; =; <>
- Logique : ET; OU; NON; OR



L'affectation

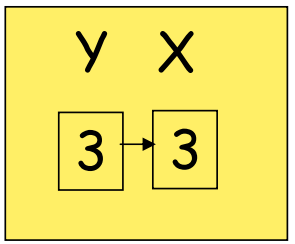
- Affecter une valeur à une variable

X:= 3 , On charge la variable X avec la valeur 3



- Affecter le contenu d'une variable à une autre variable

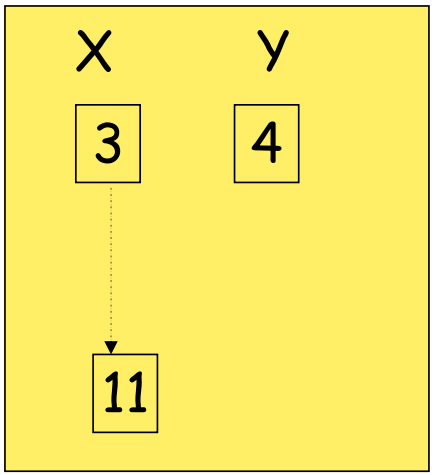
X:= Y , On charge X avec le contenu de Y



L'affectation (suite)

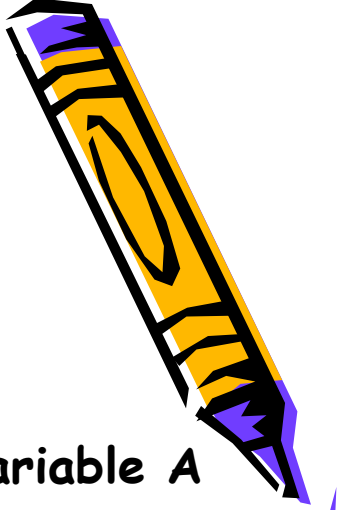
- Affecter une formule à une variable

X:= X + 2 * Y , On charge la variable X par la valeur du résultat de la formule et on écrase sa valeur initiale



X = 3 + 2 * 4

Lecture et Affichage des données



* Lecture A partir du clavier

- **Syntaxe : Lire(Variable)**
- **Exemple Lire(A) : on saisie une valeur pour la stocker après dans la variable A**

* Affichage sur Écran

- **Syntaxe : Écrire(Variable)**
- **Exemple Écrire (A) : Afficher la valeur de la variable A sur écran**



La syntaxe du pseudo-langage

Un programme comportera :

- Une partie déclaration
- Une partie encadrée par **début** **fin** où sont décrites les actions

```
programme :

déclarations;

DEBUT
                Actions
FIN
```

Dans la partie déclarations, nous trouvons :

- déclaration de variables
- déclaration de fonction

Dans la partie actions, nous trouvons :

- suite d'instructions
- alternative
- répétitive





Des Questions ?

0021266950863 :Tel

Exercice 1



Écrire un algorithme qui permet de saisir des valeurs pour A et B , faire la somme et afficher le résultat?

Solution :



Exercice 2



Écrire un algorithme qui permet de calculer et afficher la surface d'un cercle?

Solution :



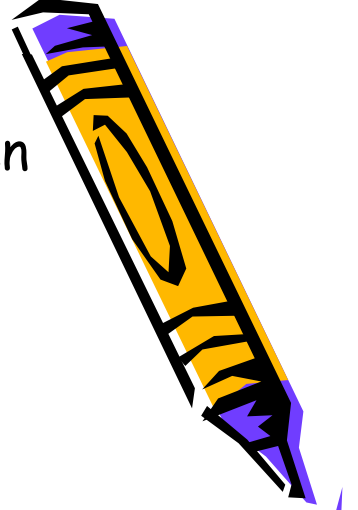
Exercice 3

Écrire un algorithme qui permet de calculer et afficher le salaire brut d'un ouvrier connaissant le nombre d'heure et le tarif d'horaire?

Solution :



Exercice 4



Écrire un algorithme qui fait la conversion d'une somme d'argent donne en DH ,en une somme d'argent en Euro?

Solution :



structure alternative

Syntaxe 1 :

```
Si condition Alors
        Actions 1
                .
                .
                .
        ActionsN
FinSi
```

Syntaxe 2 :

```
Si condition Alors
        Actions 1
                .
                .
                .
        ActionsN
Sinon
        Actions 1
                .
                .
                .
        ActionsN
FinSi
```



On peut utiliser des structures alternatives imbriquées

0021266950863 :Tel

Exercice

On désire comparer deux valeurs ,Écrire un algorithme qui affiche la plus grande des deux?

Solution :



Exercice 1



Écrire un algorithme qui affiche le salaire brut d'un ouvrier sachant que les heures supplémentaires de 172 heures sont payées à 50% de tarifs d'horaire en plus?

Solution :



Exercice 2



Afficher le résultat de fin d'année pour un étudiant connaissant sa moyenne générale?

Solution :



Exercice 3

Calculer le montant de la facture d'un client ayant commandé une quantité d'un produit avec un prix unitaire hors taxe

Le taux de T.V.A est : 20%

Les frais de transport sont 0.8 DH de Km , Le client est disposé du frais de transport si le montant est supérieur 4500 DH

Solution :



Exercice 4

Une bibliothèque fait des réductions sur l'achat des livres :

- 25% pour les étudiants.
- 15% pour les enseignants

Écrire un algorithme qui calcule et affiche le prix à payer selon le type du client?

Solution :





Des exercices supplémentaires ?

Exercice 1



Écrire un algorithme qui calcule et affiche le maximum de trois nombre A,B et C ?

Solution :



Exercice 2



Écrire un algorithme qui permet de résoudre l'équation du premier degré : aX+b=0?

Solution :



Exercice 3



Écrire un algorithme qui permet de résoudre une équation de second degré : $aX^2+bX+c=0$?

Solution :



Exercice 4



Écrire un algorithme qui lie trois nombre A,B et C , puis il détermine si l'un est égal à la somme de 2 autres sinon il affiche un message « pas de solution »?

Solution :



Exercice 5



Un patron décide de calculer le montant de la participation au prix du repas de ses employés de la façon suivante :

- S'il est célibataire : participation 20% du pris de repas
- S'il est marie : participation 25% du pris de repas.
- S'il a des enfants : participation 10% supplémentaire par enfant.

La participation est plafonnée à 50%.
Si le salaire mensuel est inférieur à 6000 DH, la participation est majorée à 10%.
Solution :



Exercice 6

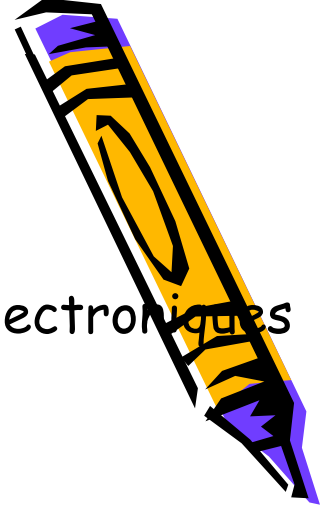


Écrire un algorithme qui à partir d'un nombre compris entre 1 et 7 affiche le jour correspendant?

Solution :



Exercice 7



Le représentant d'une société de distribution des produits d'affichage électroniques sont rémunérés de façon suivante :

- un fixe mensuel de 4000 DH
- une commission a pourcentage variable sur chaque tranche du chiffre d'affaire mensuel :
  - une commission de 5% pour un chiffre d'affaire compris entre 0 et 1000.
  - une commission de 10% pour un chiffre d'affaire compris entre 1000 et 3000.
  - une commission de 14% pour un chiffre d'affaire supérieur à 3000.

Écrire un algorithme qui permet de calculer et afficher le montant total de la rémunération de chaque représentant.



Exercice 8

Le service des prêts d'une bibliothèque à adapter le règlement suivant :

✓ tous les lecteurs de la catégories A peuvent empreinter des livres pour une durée maximale de 20 jours.

✓ un lecteur de la catégorie B peut conserver des livres pour une durée maximale de 30 jours.

✓ un lecteur de la catégorie C peut conserver des livres pour une durée maximale de 45 jours.

✓Aucun lecteur ne pourra avoir en sa possession plus de 5 ouvrages.

Écrire un algorithme qui affiche la durée d'empreinte et est ce qu'il a le droit d'empreinter ?



Structure répétitive(itérative)

Structure répétitive permet de répéter une ou plusieurs actions un certain nombre de fois. On identifie en règle générale 3 types de répétitive :

- Tant que
- Faire Jusqu'à
- Pour



structure répétitive 1



- Tant que

Les répétitives où la condition d'arrêt est placée au début.

```
TantQue    condition
           actions
FINTQ
```

Ce qui signifie : tant que la condition est vraie, on exécute les actions.



structure répétitive 2



- Faire Jusqu'à

Les répétitives où la condition d'arrêt est placée à la fin.

Faire

Actions

Jusqua **condition**

Ce qui signifie qu'on exécute les actions jusqu'à ce que la condition soit vraie.



structure répétitive 3

- Pour

Les répétitives où le nombre d'itération est fixée une fois pour toute.

Très souvent, nous utilisons une structure répétitive avec un compteur et nous arrêtons lorsque le compteur a atteint sa valeur finale.

```
POUR variable ALLANT DE entier A entier [PAS entier]

        Actions

FinPour
```



Lorsque le PAS est omis, il est supposé égal à +1.

Exercice 1

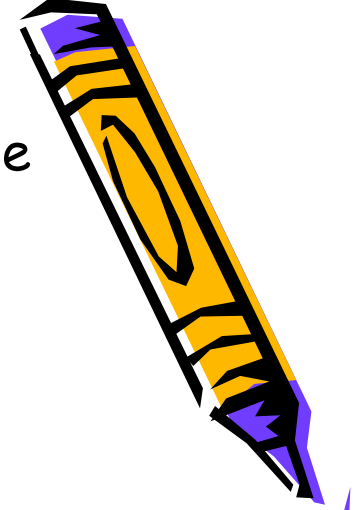


Écrire un algorithme qui saisie N entier et affiche leur somme et leur moyenne ?

Solution :



Exercice 2



Écrire un algorithme pour calculer et afficher la somme et la moyenne de 100 premiers nombres entiers ?

Solution :



Exercice 3



Écrire un algorithme qui permet de calculer S1,S2,S3,S4,S5,S6 tel que:

- **S1** = 1 + $\frac{1}{2}$ + 1/3 + $\frac{1}{4}$ +.....1/N
- **S2** = 1 + $\frac{1}{2}$ + $\frac{1}{4}$ + 1/6 +.....1/N
- **S3** = 1 + 1/3 + 1/5 +.....1/N
- **S4** = 1 - $\frac{1}{2}$ + $\frac{1}{4}$ - 1/6.....1/N
- **S5** = 1 + x+x^2 .....x^N
- **S6** = 1 + x+x^2/2..... x^N /N



Exercice 4



Écrire un algorithme qui vérifie si un nombre est premier où pas ?

Solution :



Exercice 5



écrire u algorithme qui permet de donner le plus grand commun diviseur de 2 entiers positifs A et B ?

Solution :



Exercice 6



écrire un algorithme qui permet de calculer le factoriel d'un nombre positif ?

Solution :



Exercice 7

écrire un algorithme qui saisit deux entiers, calcule et affiche la somme de ces 2 entiers (on arrête la saisie avec la valeur 0) ?

Solution :



Examen N° 1

Exercice 1 :

Une salle de cinéma désire automatiser la gestion de la billetterie pour chaque client qui se présente, on calcule le prix du billet en fonction des données suivantes:

- NF : Numéro du fil
- HS : heure de la séance
- Age : Age de spectateur

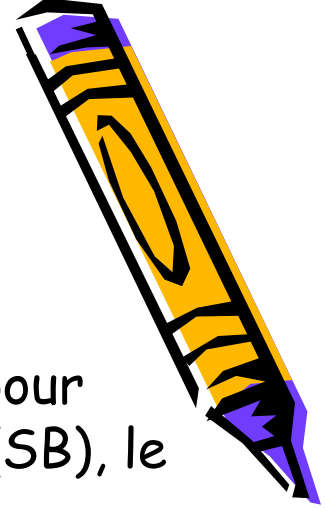
Le prix normal de la facture est de 30 DH, cependant des remises peuvent être accorder en fonction du critères suivants :

NF=2 ou Age < 15 ou HS>20 remise de 50%
NF=2 et Age >75 remise 25%
NF=3 et Age < 15 entrée refusée

Écrire un algorithme qui permet de calculer et décider le prix à payer pou chaque client qui se présente . on arrête la saisie par "non"



Examen N° 1 (suite)



Exercice 2 :

Une entreprise désire automatisée la gestion de paie de son personnel pour chaque employé, on doit introduire le nom, le prénom, le salaire de base(SB), le nombre d'enfant et l'ancienneté(ANC).

- Un prix de 100 DH est accordé pour chaque enfant .
- Si ancienneté<=10 ans et SB < 1000 DH une prime de 50% du SB
- Si l'ancienneté est 10 ans < ANC <20 ans et SB > 1000 DH prime de 70% du SB

Calculer le salaire brute (SB+ le prime + les enfants) et afficher le nom , prénom,

l'ancienneté , SB, prime et le salaire brute



Exercice

Écrire un programme qui saisit des entiers et en affiche la somme et la moyenne (on arrête la saisie avec la valeur 0)

Solution :





Des Questions ?

1. Utilité des tableaux

Imaginons que dans un programme, nous ayons besoin simultanément de 12 valeurs (par exemple, des notes pour calculer une moyenne). Évidemment, la seule solution dont nous disposons à l'heure actuelle consiste à déclarer douze variables, appelées par exemple Notea, Noteb, Notec, etc. Bien sûr, on peut opter pour une notation un peu simplifiée, par exemple N1, N2, N3, etc. Mais cela ne change pas fondamentalement notre problème, car arrivé au calcul, et après une succession de douze instructions « Lire » distinctes, cela donnera obligatoirement une atrocité du genre :

```
Moy ← (N1+N2+N3+N4+N5+N6+N7+N8+N9+N10+N11+N12)/12
```



Les tableaux



C'est pourquoi la programmation nous permet **de rassembler toutes ces variables en une seule**, au sein de laquelle chaque valeur sera désignée par un numéro. Cela donnerait donc quelque chose du genre « la note numéro 1 », « la note numéro 2 », « la note numéro 8 ». C'est largement plus pratique

Un ensemble de valeurs portant le même nom de variable et repérées par un nombre, s'appelle un tableau, ou encore une variable indicée.

Le nombre qui, au sein d'un tableau, sert à repérer chaque valeur s'appelle l'indice.

Chaque fois que l'on doit désigner un élément du tableau, on fait figurer le nom du tableau, suivi de l'indice de l'élément, entre parenthèses. Ex: ➔ Nom_tableau[5]



2. Notation et utilisation algorithmique

```
Tableau Note(12) en Numérique
Variables Moy, Som en Numérique

Début
 POUR i ALLANT DE 0 A 11
   Ecrire "Entrez la note n°", i
   Lire Note(i)
 FinPour

Som := 0

POUR i ALLANT DE 0 A 11
  Som := Som + Note(i)
 FinPour

Moy := Som / 12

Écrire(«la somme est: » Som)
Écrire(«La moyenne est: » Moy)
Fin
```



Exercice 2



Écrire un algorithme qui déclare et remplisse un tableau de 7 valeurs numériques en les mettant toutes à zéro.

Solution :



Exercice 3

Écrire un algorithme qui déclare et remplisse un tableau contenant les six voyelles de l'alphabet latin.

Solution :

```
Tableau Truc(6) en Numérique
Début

Truc(0) ← "a"
Truc(1) ← "e"
Truc(2) ← "i"
Truc(3) ← "o"
Truc(4) ← "u"
Truc(5) ← "y"

Fin
```



Exercice 4

Écrire un algorithme qui déclare un tableau de 9 notes, dont on fait ensuite saisir les valeurs par l'utilisateur.

Solution :

```
Tableau Notes(9) en Numérique
Variable i en Numérique

début

POUR i ALLANT DE 0 A 6
   Ecrire "Entrez la note numéro ", i + 1
   Lire Notes(i)
i suivant

Fin
```



Exercice 5



On saisit des entiers et on les range dans un tableau (maximum 50)
Écrire un programme qui affiche le maximum, le minimum et la valeur moyenne de ces nombres.
Solution :



Exercice 6



Écrivez un algorithme permettant à l'utilisateur de saisir un nombre quelconque de valeurs, qui devront être stockées dans un tableau. L'utilisateur doit donc commencer par entrer le nombre de valeurs qu'il compte saisir. Il effectuera ensuite cette saisie. Enfin, une fois la saisie terminée, le programme affichera le nombre de valeurs négatives et le nombre de valeurs positives.

Solution :



Solution (Exercice 5)





Des exercices supplémentaires ?

Exercice 5.1

Que produit l'algorithme suivant ?

```
Tableau Nb(6) en Entier
Variable i en Entier
Début
  Pour i ← 0 à 5
    Nb(i) ← i * i
  i suivant

  Pour i ← 0 à 5
   Écrire Nb(i)
  i suivant

Fin
```

Peut-on simplifier cet algorithme avec le même résultat ?



Exercice 5.2

Que produit l'algorithme suivant ?

```
Tableau N(7) en Entier
Variables i, k en Entier
Début

N(0) ← 1
Pour k ← 1 à 6
  N(k) ← N(k-1) + 2
k Suivant

Pour i ← 0 à 6
  Ecrire N(i)
i suivant
Fin
```

Peut-on simplifier cet algorithme avec le même résultat ?



Exercice 5.3

Que produit l'algorithme suivant ?

```
Tableau Suite(8) en Entier
Variable i en Entier

Début

Suite(0) ← 1
Suite(1) ← 1

Pour i ← 2 à 7
  Suite(i) ← Suite(i-1) + Suite(i-2)
i suivant

Pour i ← 0 à 7
  Écrire Suite(i)
i suivant
Fin
```





Exercice 5.4

Écrivez la fin de l'algorithme (exercice 5.3) afin que le calcul de la moyenne des notes soit effectué et affiché à l'écran.

Exercice 5.5

Écrivez un algorithme permettant à l'utilisateur de saisir un nombre quelconque de valeurs, qui devront être stockées dans un tableau. L'utilisateur doit donc commencer par entrer le nombre de valeurs qu'il compte saisir. Il effectuera ensuite cette saisie. Enfin, une fois la saisie terminée, le programme affichera le nombre de valeurs négatives et le nombre de valeurs positives.

Exercice 5.6

Écrivez la fin de l'algorithme 6.3 afin que le calcul de la moyenne des notes soit effectué et affiché à l'écran.



Exercice 5.7

Écrivez un algorithme calculant la somme des valeurs d'un tableau (on suppose que le tableau a été préalablement saisi).

Exercice 5.8

Écrivez un algorithme constituant un tableau, à partir de deux tableaux de même longueur préalablement saisis. Le nouveau tableau sera la somme des éléments des deux tableaux de départ.

Tableau 1 :	4	8	7	9	1	5	4	6
+								
Tableau 2 :	5	9	4	2	6	8	7	41
=								
Tableau 3 :	9	17	11	11	7	13	11	47





Des Questions ?

0021266950863 :Tel

Les Fonctions Prédéfinies



Tout langage de programmation propose ainsi un certain nombre de fonctions ; certaines sont indispensables, car elles permettent d'effectuer des traitements qui seraient sans elles impossibles. D'autres servent à soulager le programmeur, en lui épargnant de longs – et pénibles - algorithmes.

Tous les langages, je dis bien tous, proposent les fonctions suivantes, même si le nom et la syntaxe peuvent varier d'un langage à l'autre :

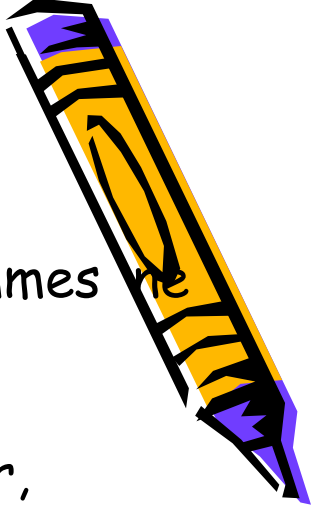
- Len(chaîne) : renvoie le nombre de caractères d'une chaîne
- Mid(chaîne,n1,n2) : renvoie un extrait de la chaîne, commençant au caractère n1 et faisant n2 caractères de long.
- Left(chaîne,n) : renvoie les n caractères les plus à gauche dans chaîne.
- Right(chaîne,n) : renvoie les n caractères les plus à droite dans chaîne
- Trouve(chaîne1,chaîne2) : renvoie un nombre correspondant à la position de chaîne2 dans chaîne1. Si chaîne2 n'est pas comprise dans chaîne1, la fonction renvoie zéro.
-Modulo : Cette fonction permet de récupérer le reste de la division d'un nombre par un deuxième nombre.



Exemples :

Len("Bonjour, ça va ?")	vaut	16
Len("")	vaut	0
Mid("Zorro is back", 4, 7)	vaut	"ro is b"
Mid("Zorro is back", 12, 1)	vaut	"c"
Left("Et pourtant…", 8)	vaut	"Et pourt"
Right("Et pourtant…", 4)	vaut	"t…«
Trouve("Un pur bonheur", "pur")	vaut	4
Trouve("Un pur bonheur", "techno")	vaut	0
A ← Mod(10,3)	A vaut 1 car 10 = 3*3 + 1	
B ← Mod(12,2)	B vaut 0 car 12 = 6*2	
C ← Mod(44,8)	C vaut 4 car 44 = 5*8 + 4	

Les fichiers

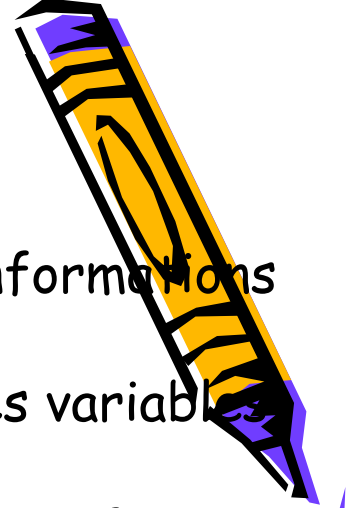


1. Utilité des fichiers

Jusqu'à présent, les informations utilisées dans nos programmes ne pouvaient provenir que de deux sources :

- soit elles étaient inclues dans l'algorithme lui-même, par le programmeur,
- soit elles étaient entrées en cours de route par l'utilisateur. Mais évidemment,
- cela ne suffit pas à combler les besoins réels des informaticiens.

Imaginons que l'on veut écrire un programme gérant un carnet d'adresses. D'une exécution du programme à l'autre, l'utilisateur doit pouvoir retrouver son carnet à jour, avec les modifications qu'il y a apportées la dernière fois qu'il a exécuté le programme. Les données du carnet d'adresse ne peuvent donc être inclues dans l'algorithme, et encore moins être entrées au clavier à chaque nouvelle exécution !



1. Utilité des fichiers

Les fichiers sont là pour combler ce manque. Ils servent à stocker des informations de manière permanente, entre deux exécutions d'un programme. Car si les variables qui sont je le rappelle des adresses de mémoire vive, disparaissent à chaque fin d'exécution, les fichiers, eux sont stockés sur des périphériques à mémoire de masse (disquette, disque dur, CD Rom…).



2. Structure des enregistrements

Reprenons le cas du carnet d'adresses, avec dedans le nom, le prénom, le téléphone et l'email. Les données, sur le fichier texte, peuvent être organisées ainsi :

Structure n°1

"Fonfec";"Sophie";0142156487;"fonfec@yahoo.fr"
"Zétofrais";"Mélanie";0456912347;"zétofrais@free.fr"
"Herbien";"Jean-Philippe";0289765194;"vantard@free.fr"
"Hergébel";"Octave";0149875231;"rg@aol.fr"

délimitée

ou ainsi :

Structure n°2

```
Fonfec      Sophie          0142156487fonfec@yahoo.fr
Zétofrais   Mélanie         0456912347zétofrais@free.fr
Herbien     Jean-Philippe   0289765194vantard@free.fr
Hergébel    Octave          0149875231rg@aol.fr
```

À champs de largeur fixe



2. Structure des enregistrements

La structure n°1 est dite délimitée ; Elle utilise un caractère spécial, appelé caractère de délimitation, qui permet de repérer quand finit un champ et quand commence le suivant.

La structure n°2, elle, est dite à champs de largeur fixe. Il n'y a pas de caractère de délimitation, mais on sait que les x premiers caractères de chaque ligne stockent le nom, les y suivants le prénom, etc. Cela impose bien entendu de ne pas saisir un renseignement plus long que le champ prévu pour l'accueillir.





2. Structure des enregistrements

L'avantage de la structure n°1 : elle occupe le minimum de place possible.

L'inconvénient : la lenteur de la lecture.

L'avantage de la structure n°2 : la récupération des différents champs est très rapide

L'inconvénient : gaspille de la place mémoire





3. Types d'accès

L'accès séquentiel : on ne peut accéder qu'à la donnée suivant celle qu'on vient de lire. Dans le cas d'un fichier texte, cela signifie qu'on lit le fichier ligne par ligne.

L'accès direct (ou aléatoire) : on peut accéder directement à l'enregistrement de son choix, en précisant le numéro de cet enregistrement. Mais cela veut souvent dire une gestion fastidieuse des déplacements dans le fichier.

L'accès indexé : pour simplifier, il combine la rapidité de l'accès direct et la simplicité de l'accès séquentiel (en restant toutefois plus compliqué). Il est particulièrement adapté au traitement des gros fichiers, comme les bases de données importantes.



4. Instructions

Si on veut travailler sur un fichier, la première chose à faire est de l'ouvrir. Cela se fait en attribuant au fichier un numéro de canal. On ne peut ouvrir qu'un seul fichier par canal. L'important est que lorsqu'on ouvre un fichier, on stipule ce qu'on va en faire : lire, écrire.

Pour ouvrir un fichier texte, on écrira par exemple :

Ouvrir « c:\Exemple.txt" **sur** 4 **en Lecture**



exemple

```
Variables Truc, Nom, Prénom, Tel, Mail en Caractères

Début
   Ouvrir "Exemple.txt" sur 4 en Lecture
   LireFichier 4, Truc

    Nom ← Mid(Truc, 1, 20)
    Prénom ← Mid(Truc, 21, 15)
    Tel ← Mid(Truc, 36, 10)
    Mail ← Mid(Truc, 46, 20)

    Ecrire( « Le nom est : »,Nom)
    Ecrire( « Le nom est : », Prénom )
    Ecrire( « Le nom est : », Tel)
    Ecrire( « Le nom est : », Mail)

   Fermer 4
Début
```

Cet algorithme affiche la première ligne d'un carnet d'adresses.

exemple

```
Variables Truc, Nom, Prénom, Tel, Mail en Caractères

Début
 Ouvrir "Exemple.txt" sur 4 en Lecture
  Tantque Non EOF(5)
    LireFichier 4, Truc
    Nom ← Mid(Truc, 1, 20)
    Prénom ← Mid(Truc, 21, 15)
    Tel ← Mid(Truc, 36, 10)
    Mail ← Mid(Truc, 46, 20)

    Ecrire( « Le nom est : »,Nom)
    Ecrire( « Le nom est : », Prénom )
    Ecrire( « Le nom est : », Tel)
    Ecrire( « Le nom est : », Mail)
  FinTantQue
 Fermer 4
Début
```

Cet algorithme affiche toutes les lignes d'un carnet d'adresses.

exemple

```
Variables Nom * 20, Prénom * 17, Tel * 10, Mail * 20, Lig en Caractère

Début
  Écrire ("Entrez le nom : ")
  Lire (Nom)
  Écrire ("Entrez le prénom : ")
  Lire (Prénom)
  Écrire ("Entrez le téléphone : ")
  Lire (Tel)
  Écrire ("Entrez le nom : ")
  Lire (Mail)
  Lig ← Nom & Prénom & Tel & Mail

  Ouvrir "Adresse.txt" sur 1 pour Ajout
  EcrireFichier 1, Lig
  Fermer 1
Fin
```



Cet algorithme permet à l'utilisateur de saisir au clavier un nouvel individu qui sera ajouté dans un carnet d'adresses.

exemple

```
Tableaux Nom(), Prénom(), Tel(), Mail() en Caractère
Début
Ouvrir "Exemple.txt" sur 5 en Lecture
i ← -1
Tantque Non EOF(5)
  LireFichier 5, Truc
  i ← i + 1
  Redim Nom(i+1)
  Redim Prénom(i+1)
  Redim Tel(i+1)
  Redim Mail(i+1)
  Nom(i) ← Mid(Truc, 1, 20)
  Prénom(i) ← Mid(Truc, 21, 15)
  Tel(i) ← Mid(Truc, 36, 10)
  Mail(i) ← Mid(Truc, 46, 20)
FinTantQue
Fermer 5
Fin
```

Cet algorithme charge toutes les lignes d'un carnet d'adresses dans 4 tableaux

Écrire un algorithme qui gère un carnet d'adresses. Puis on le codifie sous java.





Des Questions ?

Procédures et Fonctions



1. De quoi s'agit-il

Une application, surtout si elle est longue, a toutes les chances de devoir procéder aux mêmes traitements.

Par exemple, la saisie d'une réponse par oui ou par non (et le contrôle qu'elle implique), peuvent être répétés dix fois à des moments différents de la même application

La manière la plus évidente est de répéter le code correspondant autant de fois que nécessaire.

si la structure d'un programme écrit de cette manière peut paraître simple mais Elle contient des répétitions

en cas de modification du code, il va falloir traquer toutes les apparitions de ce code pour faire la modification

Il faut donc opter pour une autre stratégie, qui consiste à séparer ce traitement du corps du programme et à appeler ce traitement en cas de besoin.

il suffit de faire une seule modification au bon endroit, pour que cette modification prenne effet dans la totalité de l'application.



exemple

prenons un exemple de question à laquelle l'utilisateur doit répondre par oui ou par non.

Mauvaise Structure :

```
Ecrire "Etes-vous marié ?"

Rep1 := ""
TantQue Rep1 <> "Oui" et Rep1 <> "Non"
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep1
FinTantQue

Ecrire ("Avez-vous des enfants ?")

Rep2 := ""
TantQue Rep2 <> "Oui" et Rep2 <> "Non"
  Ecrire "Tapez Oui ou Non"
  Lire Rep2
FinTantQue
```



exemple

La solution consiste à isoler les instructions demandant une réponse par Oui ou Non, et à appeler ces instructions à chaque fois que nécessaire. Ainsi, on évite les répétitions inutiles, et on a découpé notre problème en petits morceaux autonomes.

Nous allons donc créer une fonction dont le rôle sera de renvoyer la réponse (oui ou non) de l'utilisateur. Ce mot de "fonction", ne doit pas nous surprendre : nous avons étudié précédemment des fonctions fournies avec le langage, et nous avons vu que le but d'une fonction était de renvoyer une valeur. Eh bien, c'est exactement la même chose ici, sauf que c'est nous qui allons créer notre propre fonction, que nous appellerons RepOuiNon :

```
Rep1,Rep2 en caractère

Fonction RepOuiNon() en caractère
Truc ← ""
 TantQue Truc <> "Oui" et Truc <> "Non"
  Ecrire "Tapez Oui ou Non"
  Lire Truc
 FinTantQue
Renvoyer Truc
Fin

Début
        Ecrire "Etes-vous marié ?"
        Rep1 ← RepOuiNon()
        Ecrire "Avez-vous des enfants ?"
        Rep2 ← RepOuiNon()
Fin
```





2. Passage d'arguments

Reprenons l'exemple qui précède et analysons-le. Nous écrivons un message à l'écran, puis appelons la fonction RepOuiNon pour poser une question ; puis, un peu plus loin, on écrit un autre message à l'écran, et on appelle de nouveau la fonction pour poser la même question, etc. C'est une démarche acceptable, mais qui peut encore être améliorée : puisque avant chaque question, on doit écrire un message, autant que cette écriture du message figure directement dans la fonction appelée.

Cela implique deux choses :

==> Lorsqu'on appelle la fonction, on doit lui préciser quel message elle doit afficher avant de lire la réponse

==> la fonction doit être « prévenue » qu'elle recevra un message, et être capable de le récupérer pour l'afficher.



Procédures et Fonctions

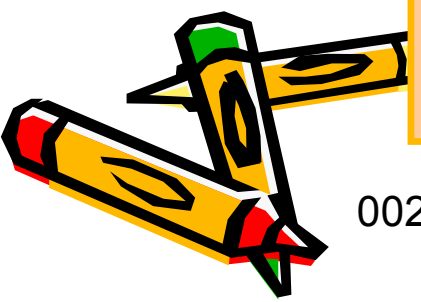
En langage algorithmique, on dira que le message devient un argument de la fonction. Cela n'est pas une découverte pour vous : nous avons utilisé les arguments à propos des fonctions prédéfinies

La fonction sera dorénavant déclarée comme suit :

```
Rep1,Rep2 en caractère

Fonction RepOuiNon(Msg en Caractère ) en caractère
Truc ← ""
Ecrire ()
TantQue Truc <> "Oui" et Truc <> "Non"
  Ecrire "Tapez Oui ou Non"
  Lire Truc
 FinTantQue
Renvoyer Truc
Fin

Début
        Rep1 ← RepOuiNon("Etes-vous marié ?" )
        Rep2 ← RepOuiNon("Avez-vous des enfants ?" )
Fin
```



```
Rep1, Rep2, Rep3, Rep4 en chaîne de caractère

début

Ecrire "Etes-vous marié(e) ?"
Lire Rep1

Ecrire ("Avez-vous des enfants ?")
Lire Rep2

Ecrire (" Etes-vous étudiant(e) ? ?")
Lire Rep3

Ecrire (" Etes-vous un homme?")
Lire Rep4
.
.
.
.
Fin
```

```
Rep1, Rep2, Rep3, Rep4 en chaîne de caractère

début

Ecrire "Etes-vous marié(e) ?"
Lire Rep1

Ecrire ("Avez-vous des enfants ?")
Lire Rep2

Ecrire (" Etes-vous étudiant(e) ?")
Lire Rep3

Ecrire (" Etes-vous un homme ?")
Lire Rep4
.
.
.
.
Fin
```



0021266950863 :Tel

```
Rep1, Rep2, Rep3, Rep4 en chaîne de caractère
début

Ecrire "Etes-vous marié(e) ?"
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep1
TantQue Rep1 <> "Oui" et Rep1 <> "Non"

Ecrire ("Avez-vous des enfants ?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep2
TantQue Rep2 <> "Oui" et Rep2 <> "Non"

Ecrire (" Etes-vous étudiant(e) ? ?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep3
TantQue Rep3 <> "Oui" et Rep3 <> "Non"

Ecrire (" Étés-vous un homme?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep4
TantQue Rep4 <> "Oui" et Rep4 <> "Non"

Fin
```

```
Rep1, Rep2, Rep3, Rep4 en chaîne de caractère
début

Ecrire "Etes-vous marié(e) ?"
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep1
TantQue Rep1 <> "Oui" et Rep1 <> "Non"

Ecrire ("Avez-vous des enfants ?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep2
TantQue Rep2 <> "Oui" et Rep2 <> "Non"

Ecrire (" Etes-vous étudiant(e) ? ?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep3
TantQue Rep3 <> "Oui" et Rep3 <> "Non"

Ecrire (" Étés-vous un homme?")
Répéter
  Ecrire ("Tapez Oui ou Non ")
  Lire Rep4
TantQue Rep4 <> "Oui" et Rep4 <> "Non"

Fin
```

```
Rep,Rep1, Rep2, Rep3, Rep4 en chaîne de
caractère

Fonction RepOuiNon() en caractère
 TantQue Rep <> "Oui" et Rep <> "Non"
   Ecrire "Tapez Oui ou Non"
   Lire Rep
 FinTantQue
Renvoyer Rep
Fin

Début
        Ecrire "Etes-vous marié ?"
        Rep1 ← RepOuiNon()

        Ecrire "Avez-vous des enfants ?"
        Rep2 ← RepOuiNon()

        Ecrire (" Etes-vous étudiant(e) ?")
        Rep3 ← RepOuiNon()

        Ecrire (" Etes-vous un homme ?")
        Rep4 ← RepOuiNon()
Fin
```

```
Rep,Rep1, Rep2, Rep3, Rep4 en chaîne de caractère

Fonction RepOuiNon() en caractère
 TantQue Rep <> "Oui" et Rep <> "Non"
   Ecrire "Tapez Oui ou Non"
   Lire Rep
 FinTantQue
Renvoyer Rep
Fin

Début
        Ecrire "Etes-vous marié ?"
        Rep1 ← RepOuiNon()

        Ecrire "Avez-vous des enfants ?"
        Rep2 ← RepOuiNon()

        Ecrire (" Etes-vous étudiant(e) ?")
        Rep3 ← RepOuiNon()

        Ecrire (" Etes-vous un homme ?")
        Rep4 ← RepOuiNon()
Fin
```

```
Rep,Rep1, Rep2, Rep3, Rep4 en chaîne de caractère

Fonction RepOuiNon(Msg en Caractère ) en caractère

Ecrire(Msg)

 TantQue Rep <> "Oui" et Rep <> "Non"
   Ecrire "Tapez Oui ou Non"
   Lire Rep
 FinTantQue
Renvoyer Rep

Fin

Début
        Rep1 ← RepOuiNon("Etes-vous marié ?" )
        Rep2 ← RepOuiNon("Avez-vous des enfants ?" )
        Rep3 ← RepOuiNon(" Etes-vous étudiant(e)? " )
        Rep4 ← RepOuiNon(" Etes-vous un homme?" )
Fin
```

Exercice 11.1

Écrivez une fonction qui renvoie la somme de trois nombres fournis en argument.

Exercice 11.2

Écrivez une fonction qui renvoie le nombre de voyelles contenues dans une chaîne de caractères passée en argument. Au passage, notez qu'une fonction a tout à fait le droit d'appeler une autre fonction.



```
Fonction RepOuiNon(val1, val2, val3 en Caractère )
en caractère

Tot = val1, val2, val3

Renvoyer Tot

Fin
```

```
Fonction NbVoyelles(Mot en Caractère)
  Variables i, nb en Numérique

  Pour i Allant de 1 à Len(Mot)

     Si Trouve("aeiouy", Mid(Mot, i, 1)) <> 0 Alors
          nb ← nb + 1
     FinSi

    FinPour

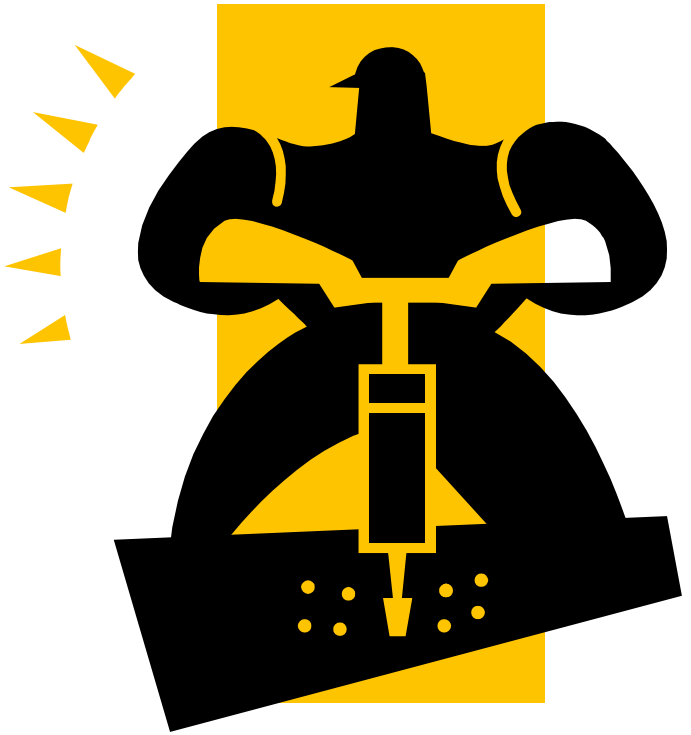
 Renvoyer nb

FinFonction
```





Des Questions ?



Des exercices supplémentaires ?

Exercice 1

Recopier une phrase dans une autre en ôtant toutes les occurrences d'un caractère
Soit une phrase terminée par un point.
Il s'agit de la restituer en supprimant les occurrences d'un caractère donné.
Exemple :

phrase :abbcccdeeeffg
caractère : c
résultat : abbdeeeffg

Donnez le jeu d'essai qui permet de tester cette procédure.
Donnez l'algorithme de la procédure en pseudo code.



Exercice 2



Recopier une phrase dans une autre en ôtant tous les doublons de caractères successifs
Soit une phrase terminée par un point.
Il s'agit de la restituer en supprimant tous les doublons de caractères successifs.
Exemple : abbcccdeeeffg. donne abcdefg.

Donnez le jeu d'essai qui permet de tester cette procédure.

Pour tester le programme, c'est à dire voir s'il répond bien à nos attentes, s'il n'a pas de "bug", avant de la faire "tourner" sur la machine nous imaginons un jeu d'essai avec tous les cas à tester et le résultat attendu pour chaque cas : **c'est le jeu d'essai.**

Donnez l'algorithme de la procédure.



Exercice 3

Déterminer si deux phrases sont équivalentes.
Soit deux phrases terminées par un même terminateur.
Elles sont dites équivalentes si elles ont les mêmes lettres dans le même ordre mais avec un nombre d'occurrences de ces lettres qui peut différer entre les deux phrases.

On supposera qu'il existe une fonction longueur **lg** de chaîne qui renvoie un entier

Exemple :

abbcccdeeeffg
aabcdeffffg sont équivalentes

Donnez le jeu d'essai qui permet de tester cette procédure.
Donnez l'algorithme de la procédure toujours en pseudo code.



Exercice 4

Chercher les lettres d'un mot éparpillées dans une phrase, dans le même ordre.

Soient un caractère terminateur et une phrase terminée par ce caractère terminateur.
Soient un mot donné
Il s'agit de vérifier si les lettres du mot sont bien présentes dans la phrase, ce dans le même ordre que celui du mot.
Exemple :

terminateur : .
phrase :le chat est gris et boit.
mot : lattis
longueur : 6
donne vrai

Donnez l'algorithme de la procédure en pseudo code.



Exercice 5

Effectuer la saisie d'une chaîne de caractères qui contiendra un nom et un prénom. Les prénoms composés seront obligatoirement séparés par des tirets.

Afficher une chaîne de caractères sous forme prénom nom séparés par un espace, en ayant fait disparaître les tirets saisis dans le prénom.

Ecrire la procédure en pseudo-code (éventuellement ensuite avec un langage).

Ne pas utiliser les instructions de type concaténation et recherche d'un caractère dans une chaîne.



Exercice 6



Déterminer si une chaîne de caractères est un palindrome.
Un palindrome est une phrase qui peut se lire dans les deux sens.
Les espaces sont ignorés.
Exemple : esope reste ici et se repose.

Le terminateur est ici un point.

Donnez l'algorithme du programme.



Exercice 7

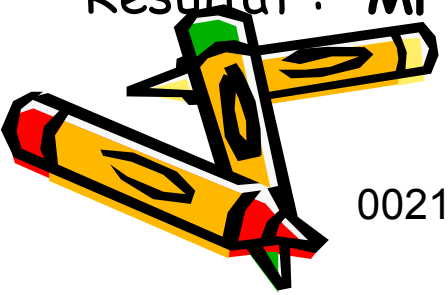
Crypter une phrase en codant les lettres en fonction du mot où elles se trouvent
Soit une phrase terminée par un point.

Les espaces sont des séparateurs de mot et sont transcrits sans modification.
Il s'agit de la crypter en codant chaque mot suivant son rang dans la phrase.
Au mot de rang 1, on crypte ses lettres avec les lettres qui suivent dans l'alphabet.
Au mot de rang 2, on crypte ses lettres avec les lettres qui suivent de 2 caractères dans l'alphabet. etc ...

Par convention, la lettre suivant le caractère Z est le caractère A.

Les espaces sont des séparateurs de mot et sont transcrits sans modification.

Exemple :
Phrase : **LE CHAT EST GRIS .**
Rang : 1 2 3 4
Résultat : **MF EJCV HVW KVMW.**



Exercice 8

Compter le nombre de mots d'une phrase ayant une terminaison donnée.
Soit une phrase terminée par un point.
Les espaces sont des séparateurs de mot.

Il s'agit de donner le nombre de mots de la phrase ayant pour terminaison la chaîne intitulée terminaison.
Exemple : Caractère final : .
Phrase : rien ne sert de courir il faut partir à point il ne faut pas rire.
Terminaison : rir
Résultat : 1

Note : les terminaisons de longueur nulle indiquent à la procédure qu'il faut renvoyer le nombre de mots de la phrase.

Ecrire la procédure en pseudo code



Exercice 9

Le programme réalise l'addition de deux données exprimées en **HH :MM:SS** et affiche le résultat sous la même forme.

Exercice 10

Écrire le programme du jeu du pendu.
Le principe est le suivant :
Un premier joueur choisit un mot de moins de 10 lettres.
Le programme affiche _ _ _ _ _ _ _ _ _ avec un _ par lettre.
Le deuxième joueur propose des lettres jusqu'à ce qu'il ait trouvé le mot ou qu'il soit pendu (11 erreurs commises).

A chaque proposition le programme réaffiche le mot avec les lettres découvertes ainsi que les lettres déjà annoncées et le nombre d'erreurs.

En deuxième partie:
Réécrire le jeu du pendu en utilisant des fonctions et/ou procédures.

